pfi

Python for Fixed Income

COURSE 5B · TIME SERIES MODELS

Missing Data, Gaps, and Corrections

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Course 5B: a series, its gaps, and its memory

Week 1: A series and its memory

Gaps, baselines, stationarity, ACF, ARIMA

Week 2: Beyond ARIMA

Smoothing, GARCH, lag features, two series

Today fits nothing: it describes and repairs, so it works on the whole file.

Today's question

Where is every gap in a dated table, what kind is it, and what does each kind need?

Every model in this course needs a value on every date it is fitted on. A real series never arrives that way.

Find it, diagnose it, correct it with a record, fill it on purpose.

Names for a repaired table

A model's output in this course is a fitted change or a one-step value for a date inside the file. Today's version:

Allowed names

a filled value, with its method

a corrected value, in the change log

a blank, left visible

Never called

a market print or an observed yield

the true value

data, once it was filled

Once a gap is filled it is easy to forget it was a gap. The names are how you do not forget.

Four kinds of missing

Kind	How it looks in pandas	Published file	Planted copy
An absent row	the date is not in the index	126	126
A blank cell	NaN in a row that has other values	0	295
Not started yet	NaN in every row before the series' first value	5,432	5,432
A wrong value	a number, the wrong one	0	3

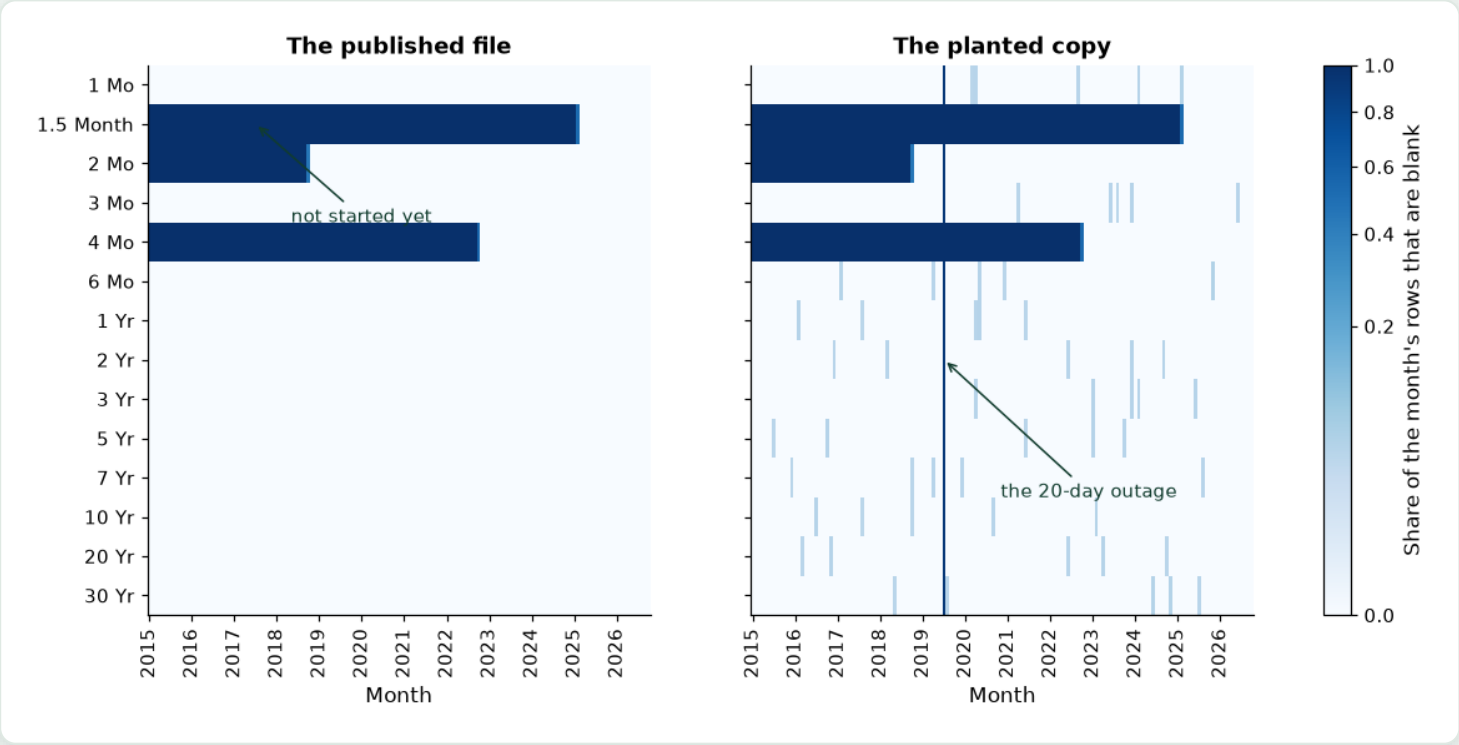
3,066 weekdays in the span, 2,940 rows in the file: 126 weekdays have no row. The planted copy adds an outage from 3 Jul 2019 to 31 Jul 2019, single blank days, and three wrong values.

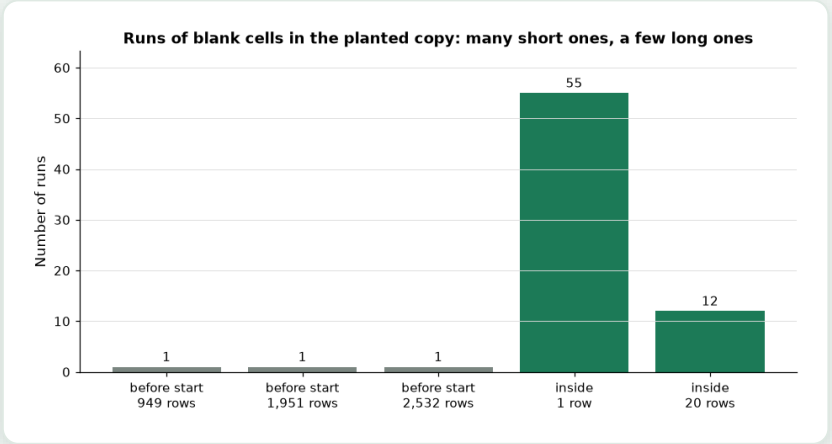
The wrong value is the dangerous one: isna cannot see it.

How gap runs work

The problem	A count of blanks says how many, not where or how long; one day and 20 days need different repairs
Mark and number	Mark each row blank or not, and start a new run number wherever that mark changes
Each run	Its first and last date, its length in rows, and a kind: before start, inside, or after end
gap_report	One row per column: where the series starts and ends, and the blanks before, inside, after
What comes out	Two small tables a person can read and a later check can assert on

Every gap at a glance





70 runs, five lengths: 55 single days, 12 runs of 20, three long runs before a start.

Draw the heatmap first: it shows which gaps are structural and which are accidents.

Diagnosing the absent weekdays

117

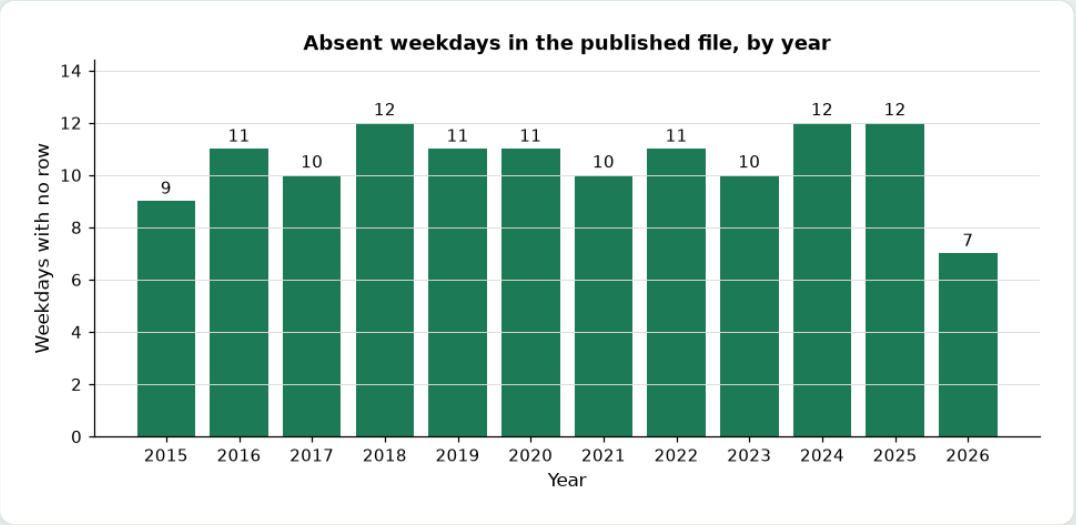
federal holidays (pandas' calendar)

8

Good Fridays

1

neither: 5 Dec 2018



The other way round: on 4 Fridays that pandas marks as a holiday (a Saturday holiday moved), the file has a row.

A calendar is where the question starts. The file's own dates are the answer.

Which repair each gap needs

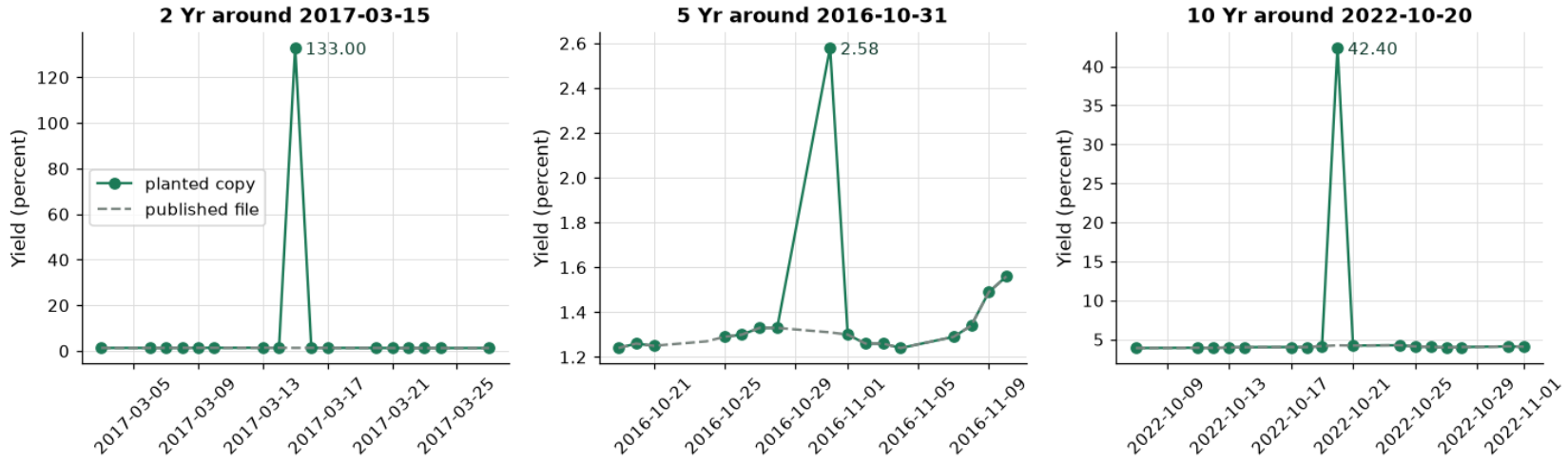
Gap	What to do	Never
An absent row	leave the date out	add the row and fill it
Not started yet	start the series at its first value	backfill it
A short outage	forward fill, or interpolate to describe	fill a scored date from a later one
A long outage	a second source; else leave it blank	interpolate across it
A wrong value	correct it from a source, and log it	overwrite it with no record

The last column is where most real mistakes come from.

How corrections and the change log work

The problem	A wrong value is the one kind isna cannot find
Find it	By its shape (a jump that comes straight back) or by a second source that disagrees
The table	One row per cell: date, column, new value (NaN blanks it), and where it came from
Apply and log	apply_corrections writes each row into a copy and logs date, column, old, new, source
What comes out	The corrected table and a change log to save and commit with it

A wrong print jumps and comes back



2 Yr 2017-03-15: 133.00 | 5 Yr 2016-10-31: 2.58 | 10 Yr 2022-10-20: 42.40

A plain 50 bp rule flags 6 real moves in the published file. Jump and come straight back: 0 in the published file, the 3 planted in the copy. The second source covers 2,874 cells and disagrees on 1.

A flag is a question to check against a source, never a correction by itself.

The update trap, and the change log

DataFrame.update skips NaN

```
updated.update(table)
```

The 5 Yr on 31 Oct 2016: 2.58 became 2.58. A blank in the table means "this is wrong", and update ignores it.

date	column	old	new	source
2016-10-31	5 Yr	2.58	blank	blanked: 30 Yr's yield
2017-03-15	2 Yr	133.00	1.33	bp: divided by 100
2022-10-20	10 Yr	42.40	4.24	desk's sheet
2022-06-17	2 Yr		3.17	second source
2023-02-06	10 Yr		3.63	second source
2023-12-05	2 Yr		4.57	second source
2024-09-18	2 Yr		3.61	second source

apply_corrections writes the blank too, and logs every change. 7 tests pass.

How a fill works

The problem	A blank cell needs a number from the values around it in the same column
Copy	ffill copies the last value before the blank down; bfill copies the first value after it up
Draw a line	interpolate reads each blank off a straight line between the two ends: by rows, or by days with time
You control	How far a fill reaches (limit) and where it may fill (limit_area)
What comes out	A table with fewer blanks, in which every filled cell is a guess

Every setting of interpolate, ffill, and bfill

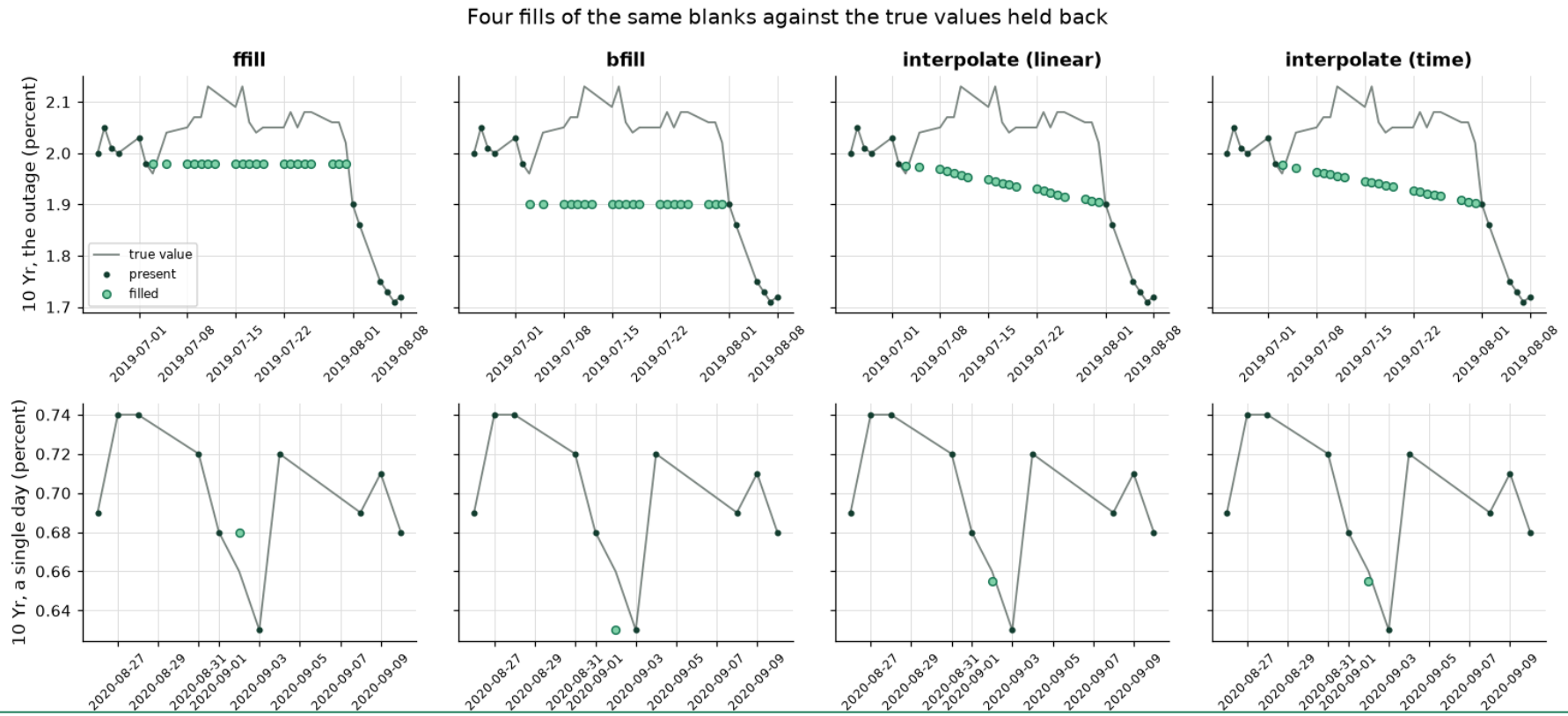
Setting	Default	Tune first
method	'linear'	yes
limit	None	yes
limit_area	None	yes
limit_direction	None	no
axis	0	no
inplace	False	no

The three that matter

method: linear counts rows, time counts days. limit: the most blanks in a row; the default None is no limit at all. limit_area: inside fills only between two values. Defaults read from the installed pandas.

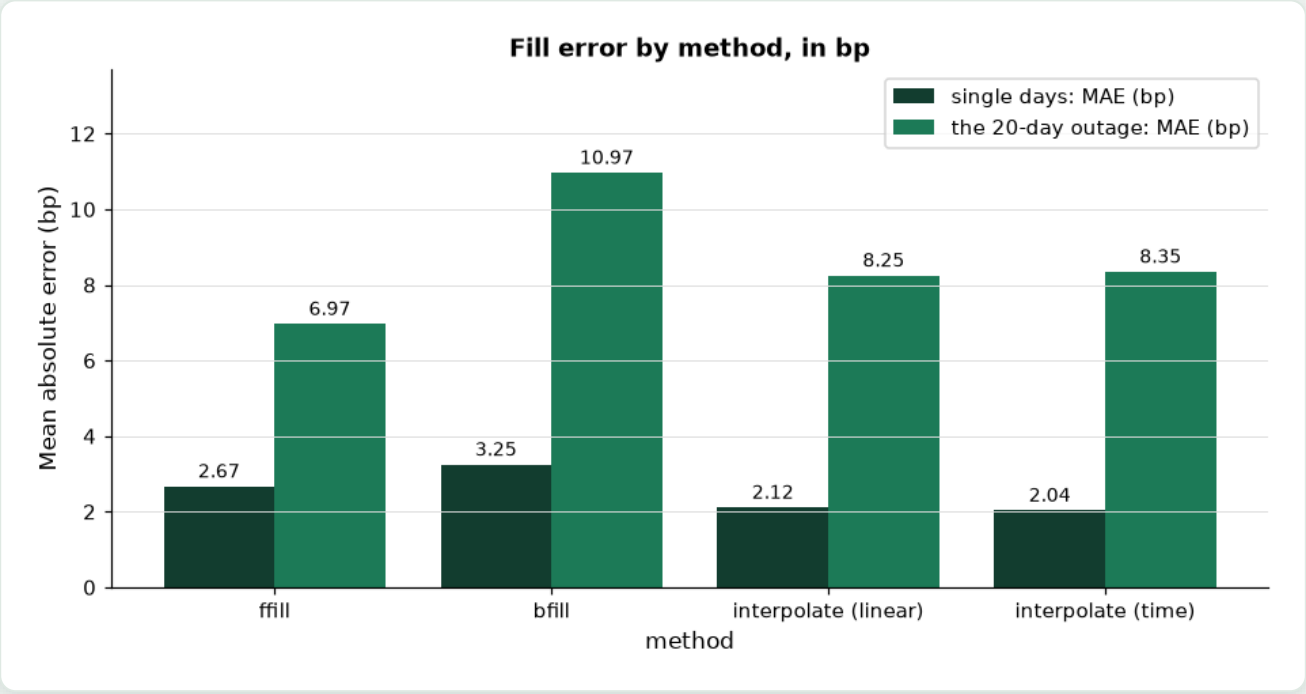
limit_direction stays forward: backward reaches into the past from the future.

Four fills, side by side



No fill recovers 20 days nobody recorded. Across one day, a line lands close; a copy lands a day's move away.

Fill error, in basis points

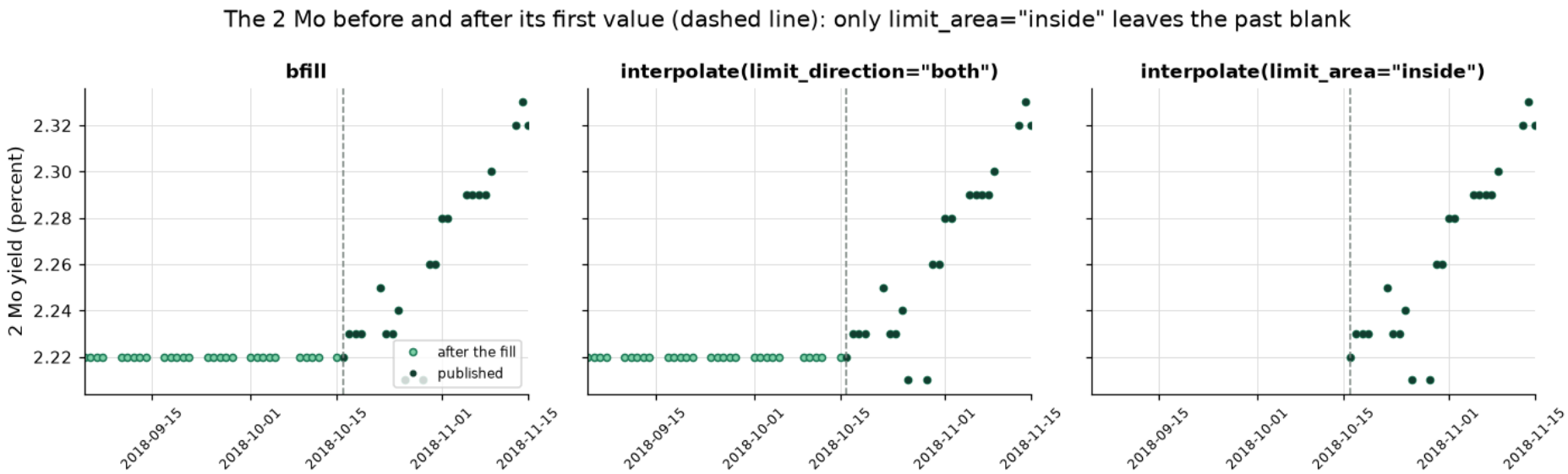


Method	1 day	Outage	Worst
ffill	2.67	6.97	20.00
bfill	3.25	10.97	23.00
interpolate (linear)	2.12	8.25	18.43
interpolate (time)	2.04	8.35	18.73

1 day and Outage: MAE in bp; Worst: the largest outage miss. bfill also filled 5,432 cells before a series started.

Across 20 days no method is reliably closer, and the lowest error does not make a method right.

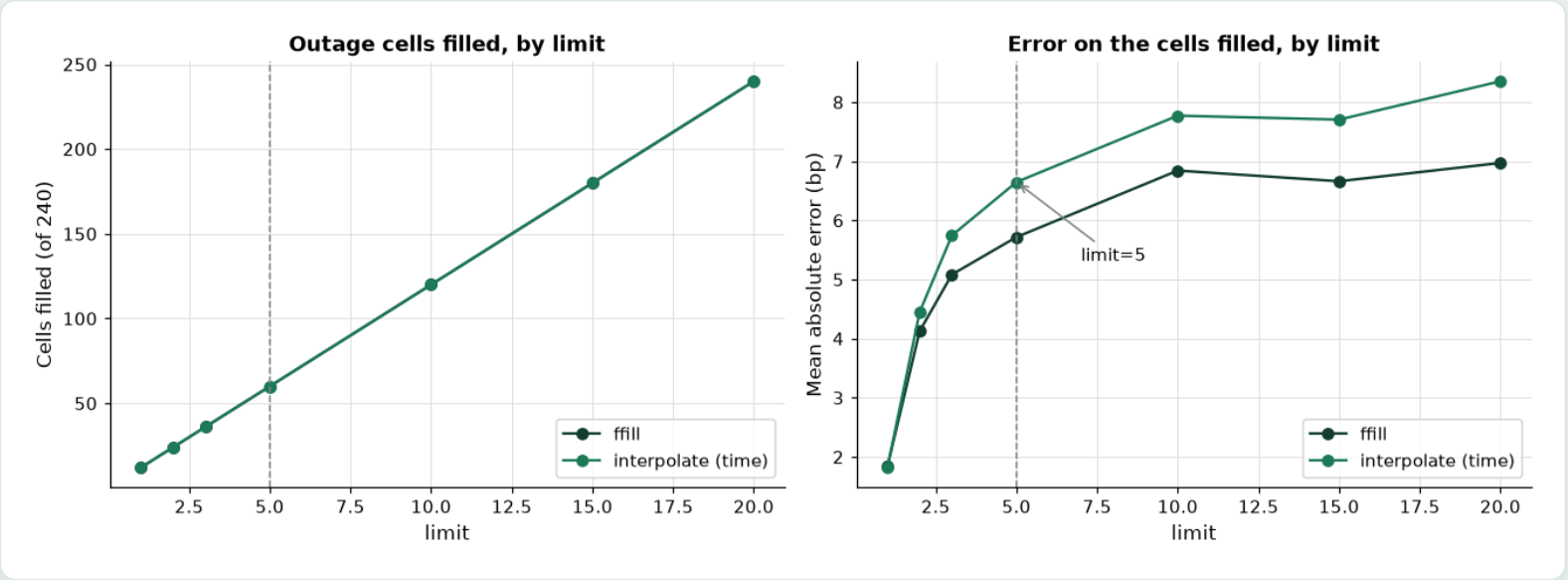
Where a fill may reach: limit_area



Before 16 Oct 2018: bfill and limit_direction="both" each fill 949 cells; limit_area="inside" fills 0.

Pass limit_area="inside" whenever a table holds a late starter.

How far a fill should reach: limit



Limit	ffill	time
1	1.83	1.82
5	5.72	6.65
10	6.84	7.77
20	6.97	8.35

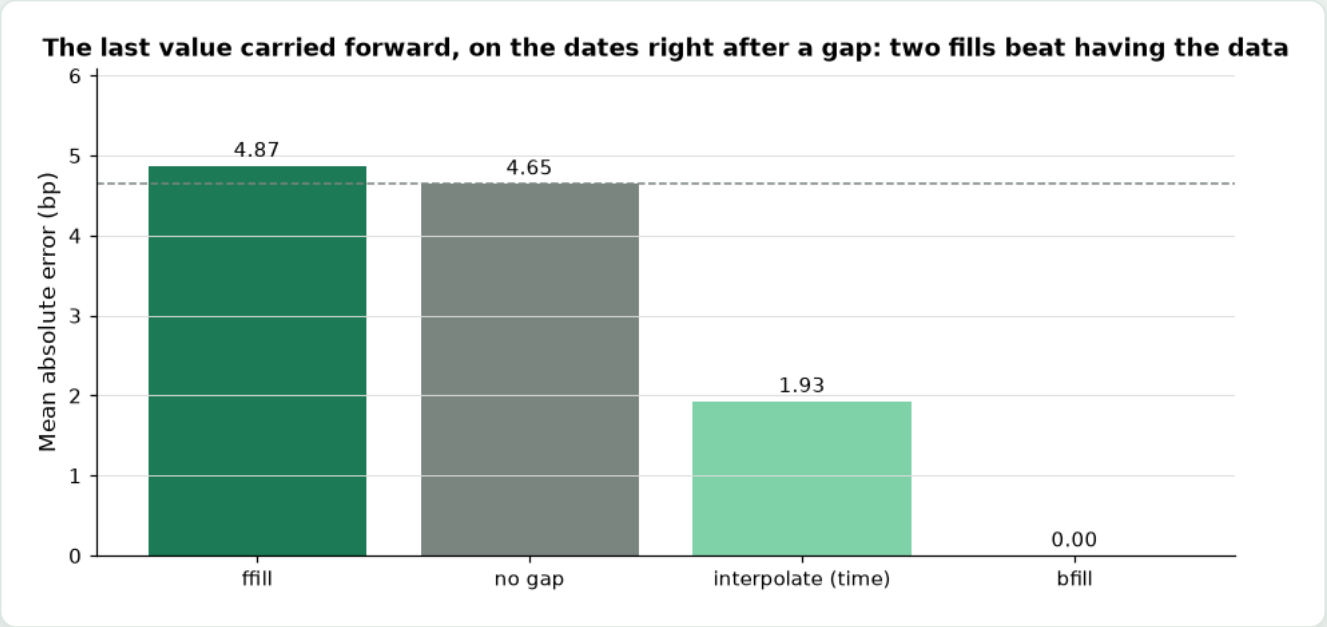
MAE in bp on the outage cells each limit fills.

Real work never shows the right panel: choose the limit from what a stale value may cost. Day 1 uses 5.

Which fill when, and a false gain

ffill	a quote you could have traded on
interpolate	a curve point, or a description
never	a later value in a scored series

On the 63 dates after a gap: bfill 0.000 bp, time interpolation 1.930, the file with no gap 4.651, ffill 4.873.

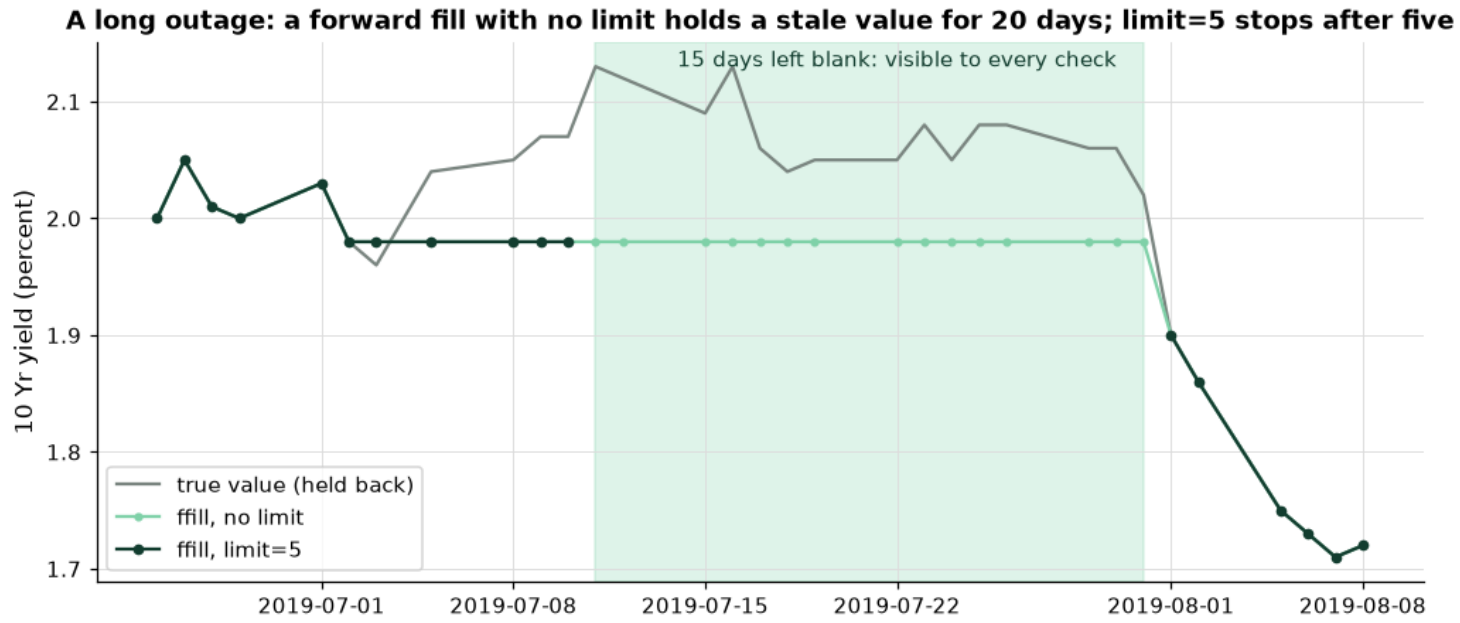


A gap cannot make a rule better than having the data. A bar below the line used a later value.

How a limit works

The problem	A fill with no limit carries a stale value, or draws a line, across any gap however long
The cap	Within each run, the first limit blanks are filled and the rest stay blank
You control	The limit, from how stale a value may be before it misleads: five trading days here
Not a skip	A limit fills the first rows of a long run; to leave a run blank, mark it with gap_runs
What comes out	Short gaps filled, long gaps shorter and still visible to every check

A long outage, left visible



limit=5

15 blank days left, 11 Jul 2019 to 31 Jul 2019. The check that stops raises; then 2,925 of 2,940 dates are scored and 15 left out, said in the output.

Leaving dates out is fine when you say so. Quietly scoring invented days is not.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTBLANK(B2:B2941)`	25 blank cells
`=NETWORKDAYS(A2,A2941)-ROWS(A2:A2941)`	126 absent weekdays
`=IF(B3="",C2,B3)`	forward fill, filled down
`=$B$1127+($B$1148-$B$1127)*(ROW()-ROW($B$1127))/(ROW($B$1148)-ROW($B$1127))`	by rows: equal within 1e-12
`=$B$1127+($B$1148-$B$1127)*($A1128-$A$1127)/($A$1148-$A$1127)`	by dates: equal within 1e-12
`=IF(G2="",XLOOKUP(A2,backup!$A$2:$A$1440,backup!$B$2:$B$1440,""),G2)`	combine_first, every row
`=IFERROR(AND(ABS(I4)>50,ABS(I3)>50,SIGN(I4)=-SIGN(I3)),FALSE)`	TRUE on 2022-10-21 only

Excel has no gap-fill function and no change log: each step is a formula you can see.

Save the day: code and the change log

1. Init

The work folder, local settings, the guard

2. .gitignore

__pycache__/, .pytest_cache/, *.xlsx

3. Code

mlkit.py, tskit.py, and its tests

4. Change log

changes.csv, in its own commit

```
Day 1: the change log of the planted copy
Day 1: tskit with gap_report, gap_runs, and apply_corrections
```

A correction to a dataset is a change like any other: it gets a commit and a message.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which dates fit, which dates score, what may be filled: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: interpolate with no limit fills 20 of 20 outage days: 0.22 bp of daily spread, not 4.34.

Code that runs is not the same as code that is right.

What you can do now

- Tell an absent row, a blank, a late start, and a wrong value apart
- Find every gap with a heatmap, gap_runs, and gap_report
- Correct with a table and a change log; update cannot blank a cell
- Choose a fill, a limit, and limit_area on purpose
- Spot a score that a fill improved falsely

Week 1, Day 2 Frequency, lags, and the naive baselines